План выполнения индивидуального задания ПП.02.01

Оглавление

1. Анализ индивидуального задания.....6
2. CSV-файл с исходными данными для варианта 1 «Электроника».....7
sample_products.csv.....7
2.1. Дополнительные тестовые CSV-файлы для модульных тестов.....8
valid_electronics.csv.....8
invalid_price.csv.....8
duplicate_sku.csv.....9
missing_brand.csv.....9
warranty_out_of_range.csv.....10
bad_active.csv.....10
empty_file.csv.....11
3. Подробный план реализации проекта ECommerceCsvImport.....11
3.1. Целевая структура проекта.....11
3.2. Технологический стек.....12
3.3. Модель товара для варианта 1 «Электроника».....13
3.4. Модель результата импорта.....14
3.5. Интерфейс репозитория в памяти.....15
3.6. Реализация репозитория в памяти.....15
3.7. Интерфейс сервиса импорта.....16
3.8. Правила валидации CSV для варианта 1 «Электроника».....17
Таблица валидации.....17
3.9. Поведение сервиса импорта.....18
Алгоритм ImportFromCsv.....18
Алгоритм ImportFromLines.....19
11.1. Пустые строки.....20

11.2. Непустая строка данных.....20
11.3. Извлечение значений по колонкам.....21
11.4. Валидация полей.....21
11.5. Если есть ошибки валидации.....23
11.6. Проверка дубликата SKU.....24
11.7. Создание объекта Product.....24
11.8. Добавление товара в репозиторий.....25
Финальная обработка результата.....25
3.10. Парсер CSV-строк.....26
3.11. Контроллер ProductsController.....27
3.12. View-модели.....29
ImportViewModel.....29
ProductsIndexViewModel.....29
3.13. Представление Views/Products/Import.cshtml.....29
3.14. Представление Views/Products/Index.cshtml.....31
4. План тестирования.....32
4.1. Тесты репозитория.....32
Тест 1: Добавление товара.....32
Тест 2: Поиск по SKU.....33
Тест 3: Поиск по SKU без учета регистра и пробелов.....33
Тест 4: Пустой репозиторий.....34
4.2. Тесты парсера CSV.....34
Тест 1: Простая строка без кавычек.....34
Тест 2: Пустые поля.....34
Тест 3: Поле с запятой внутри кавычек.....35
Тест 4: Экранированные кавычки.....35
4.3. Тесты сервиса импорта.....35

Тест 1: Успешный импорт одной корректной строки.....35
Тест 2: Успешный импорт нескольких строк.....36
Тест 3: Файл не найден.....36
Тест 4: Пустой файл.....37
Тест 5: Только заголовок без данных.....37
Тест 6: Неверный заголовок CSV.....38
Тест 7: Неверное количество полей в строке данных.....38
Тест 8: Пустой SKU.....38
Тест 9: Пустое название товара.....39
Тест 10: Неверная цена — отрицательное значение.....39
Тест 11: Неверная цена — нечисловое значение.....40
Тест 12: Отрицательный остаток.....40
Тест 13: Неверное значение is_active.....41
Тест 14: Отсутствует бренд.....41
Тест 15: Гарантия меньше нуля.....42
Тест 16: Гарантия больше 60.....42
Тест 17: Дубликат SKU внутри одного файла.....42
Тест 18: Дубликат SKU с уже существующим товаром в репозитории 43
Тест 19: Несколько ошибок в одной строке.....44
Тест 20: Пустые необязательные поля разрешены.....44
Тест 21: Пустая валюта заменяется на RUB.....45
Тест 22: Пустая категория заменяется на «Без категории».....45
Тест 23: Кириллица сохраняется корректно.....45
4.4. Ручные тестовые сценарии для UI.....46
Сценарий 1: Успешный импорт.....46
Сценарий 2: Неверная цена.....46

Сценарий 3: Дубликат SKU....46
Сценарий 4: Пустой файл....47
Сценарий 5: Неверное значение is_active....47
5. Пошаговый план разработки проекта....47
Шаг 1. Создать решение и проекты....47
Шаг 2. Добавить CSV-файлы....48
Шаг 3. Реализовать модели данных....48
Шаг 4. Реализовать репозиторий в памяти....48
Шаг 5. Реализовать парсер CSV-строк....49
Шаг 6. Реализовать сервис импорта....49
Шаг 7. Зарегистрировать зависимости....50
Шаг 8. Реализовать контроллер....50
Шаг 9. Реализовать представления....51
Шаг 10. Провести ручное тестирование....51
Шаг 11. Выполнить самопроверку качества кода....51
Шаг 12. Оформить отчет по практике....52
6. Рекомендуемые Git-коммиты....53
7. Финальный чек-лист готовности задания к защите....53
8. Краткое резюме для реализации....54

1. Анализ индивидуального задания

- специальность **09.02.07 «Информационные системы и программирование»**;
- квалификация — **Программист**;
- технология — **C# / ASP.NET MVC**;
- срок выполнения — **1 неделя**;
- итоговая аттестация — **дифференцированный зачет**.

Суть задания:

> Разработать ASP.NET MVC-модуль импорта товаров в электронный магазин из CSV-файла с хранением данных в коллекциях в памяти.

Основные требования к модулю:

1. Чтение CSV-файла построчно.
2. Преобразование строк CSV в объекты C#.
3. Валидация данных.
4. Добавление корректных товаров в память приложения.
5. Отображение результата импорта:
    - всего строк;
    - успешно импортировано;
    - количество ошибок;
    - список ошибок с номерами строк.
6. Страница просмотра списка товаров.
7. Отладка, тестирование, Git-репозиторий и отчет по практике.

Вариант 1 «Электроника» расширяет базовую модель товара дополнительными полями:

Поле	Описание	Требование
`brand`	Бренд	обязательно
`warranty_months`	Гарантия, месяцев	целое число от 0 до 60
`model_number`	Модель	может быть пустым

Для реализации я буду использовать следующую интерпретацию:

- хранение товаров — в памяти через singleton-репозиторий;

- дубликаты SKU — пропускаются, добавляется ошибка/предупреждение;
- CSV читается без внешних библиотек, но с поддержкой кавычек и запятых внутри полей;
- тесты — xUnit для репозитория, сервиса импорта и парсера CSV;
- UI — ASP.NET MVC страницы `Import` и `Index`.

---

2. CSV-файл с исходными данными для варианта 1 «Электроника»

Сохраните этот файл как:

`wwwroot/csv/sample_products.csv`

Рекомендуемая кодировка: **UTF-8**, желательно **с BOM**, чтобы корректно отображалась кириллица.

Все строки ниже являются корректными. Ожидаемый результат импорта этого файла:

Показатель	Значение
Всего строк	11
Успешно импортировано	11
Ошибки	0

`sample_products.csv`

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad 3-15
1002,Мышь Logitech M185,Беспроводная мышь с USB-приемником,1290,RUB,Периферия,10,/images/mouse.jpg,true,Logitech,12,M185
1003,Клавиатура A4Tech Bloody V3M,Механическая игровая клавиатура,3490,RUB,Периферия,7,/images/keyboard.jpg,false,A4Tech,24,V3M
1004,Монитор Dell S2722QC,Игровой монитор 27 дюймов,29990,RUB,Компьютеры,3,/images/monitor.jpg,true,Dell,24,S2722QC
1005,Видеокарта NVIDIA GeForce RTX 3060,Графическая карта для игр и работы,24990,RUB,Компьютеры,4,/images/gpu.jpg,true,NVIDIA,36,RTX3060
1006,Накопитель SSD Samsung 870 EVO 1TB,Внутренний твердотельный накопитель,5990,RUB,Компьютеры,12,/images/ssd.jpg,true,Samsung,60,870EVO-1T
1007,Webcam Logitech C920,Веб-камера Full
```

```
HD,4990,RUB,Периферия,6,/images/webcam.jpg,true,Logitech,24,C920
1008,Аудиоинтерфейс Focusrite Scarlett 2i2,Для записи звука и
подкастов,15990,RUB,Звук,2,/images/audio.jpg,false,Focusrite,60,Scarle
tt2i2
1009,Роутер TP-Link Archer AX55,Беспроводной роутер Wi-Fi
6,7490,RUB,Сетевое оборудование,8,/images/router.jpg,true,TP-
Link,36,AX55
1010,Игровая гарнитура HyperX Cloud Stinger,Наушники с
микрофоном,3290,RUB,Периферия,15,/images/headset.jpg,true,HyperX,12,Cl
oudStinger
1011,Блок питания Corsair
CV550,,4990,RUB,Компьютеры,9,,true,Corsair,60,CV550
```

2.1. Дополнительные тестовые CSV-файлы для модульных тестов

Эти файлы можно положить в папку:

`tests/ECommerceCsvImport.Tests/testdata/`

или создавать во временной директории внутри тестов.

`valid_electronics.csv`

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad
3-15
1002,Мышь Logitech M185,Беспроводная мышь с USB-
приемником,1290,RUB,Периферия,10,/images/mouse.jpg,true,Logitech,12,M1
85
```

Ожидаемый результат:

Показатель	Значение
Всего строк	2
Успешно импортировано	2
Ошибки	0

`invalid_price.csv`

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,brand,warranty_months,model_number
1003,Клавиатура A4Tech Bloody V3M,Механическая игровая клавиатура,-
500,RUB,Периферия,7,/images/keyboard.jpg,true,A4Tech,24,V3M
```

Ожидаемый результат:

Показатель	Значение
Всего строк	1
Успешно импортировано	0
Ошибки	1

Сообщение об ошибке должно содержать:

```
Поле price должно быть числом больше 0
```

Номер строки — 2, если первая строка является заголовком.

---

duplicate_sku.csv

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad
3-15
1001,Ноутбук ASUS VivoBook,Другой
ноутбук,52990,RUB,Компьютеры,7,/images/asus.jpg,true,ASUS,24,VivoBook
```

Ожидаемый результат:

Показатель	Значение
Всего строк	2
Успешно импортировано	1
Ошибки	1

Сообщение об ошибке должно содержать:

```
Артикул товара уже существует
```

В репозитории должен остаться только первый товар с SKU 1001.

---

missing_brand.csv

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,brand,warranty_months,model_number
1005,Видеокарта NVIDIA GeForce RTX
3060,,24990,RUB,Компьютеры,4,/images/gpu.jpg,true,,36,RTX3060
```

Ожидаемый результат:

Показатель	Значение
Всего строк	1
Успешно импортировано	0

Показатель	Значение
Ошибки	1

Сообщение об ошибке должно содержать:

```
Поле brand обязательно для варианта «Электроника»
```

---

warranty_out_of_range.csv

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1006,Накопитель SSD Samsung 870 EVO 1TB,Внутренний твердотельный накопитель,5990,RUB,Компьютеры,12,/images/ssd.jpg,true,Samsung,61,870EVO-1T
```

Ожидаемый результат:

Показатель	Значение
Всего строк	1
Успешно импортировано	0
Ошибки	1

Сообщение об ошибке должно содержать:

```
Гарантия должна быть целым числом от 0 до 60 месяцев
```

---

bad_active.csv

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1007,Webcam Logitech C920,Веб-камера Full HD,4990,RUB,Периферия,6,/images/webcam.jpg,yes,Logitech,24,C920
```

Ожидаемый результат:

Показатель	Значение
Всего строк	1
Успешно импортировано	0
Ошибки	1

Сообщение об ошибке должно содержать:

```
Поле is_active должно иметь значение true, false, 1 или 0
```

`empty_file.csv`

Файл должен быть пустым, то есть содержать 0 байт.

Если создадите его через редактор, убедитесь, что в нем нет невидимых символов.

Ожидаемый результат:

Показатель	Значение
Всего строк	0
Успешно импортировано	0
Ошибки	1

Сообщение об ошибке должно содержать:

`Данные для импорта отсутствуют`

---

3. Подробный план реализации проекта

`ECommerceCsvImport`

Ниже приведена детальная спецификация, по которой можно реализовать проект и написать тесты без дополнительных допущений.

3.1. Целевая структура проекта

Рекомендуемая структура:

```
ECommerceCsvImport/
├── Controllers/
│   └── ProductsController.cs
├── Models/
│   ├── Product.cs
│   ├── ImportResult.cs
│   ├── ImportError.cs
│   ├── ImportViewModel.cs
│   └── ProductsIndexViewModel.cs
├── Services/
│   ├── IProductRepository.cs
│   ├── InMemoryProductRepository.cs
│   ├── IProductImportService.cs
│   ├── ProductImportService.cs
│   └── CsvLineParser.cs
├── Views/
│   ├── Shared/
```

```
│   │   └── _Layout.cshtml
│   └── Products/
│       ├── Index.cshtml
│       └── Import.cshtml
├── wwwroot/
│   └── csv/
│       └── sample_products.csv
├── Program.cs
└── ECommerceCsvImport.csproj

ECommerceCsvImport.Tests/
├── RepositoryTests.cs
├── CsvLineParserTests.cs
├── ProductImportServiceTests.cs
├── testdata/
│   ├── valid_electronics.csv
│   ├── invalid_price.csv
│   ├── duplicate_sku.csv
│   ├── missing_brand.csv
│   ├── warranty_out_of_range.csv
│   ├── bad_active.csv
│   └── empty_file.csv
└── ECommerceCsvImport.Tests.csproj
```

Если хотите более чистую архитектуру, можно вынести модели и сервисы в отдельный class library:

```
src/ECommerceCsvImport.Core/
tests/ECommerceCsvImport.Tests/
src/ECommerceCsvImport.Web/
```

Но для учебного задания достаточно разместить все внутри ASP.NET MVC-проекта.

3.2. Технологический стек

Рекомендуется использовать:

Компонент	Рекомендация
Язык	C#
Платформа	.NET 8 или .NET 6+
Фреймворк	ASP.NET Core MVC
Тесты	xUnit
Система контроля версий	Git
Внешние CSV-библиотеки	Не обязательны, лучше реализовать собственный парсер

Важно:

> Репозиторий товаров должен быть зарегистрирован как **singleton**, потому что данные хранятся в памяти и должны сохраняться между HTTP-запросами.

Пример регистрации зависимостей:

```
builder.Services.AddSingleton<IProductRepository,
InMemoryProductRepository>();
builder.Services.AddSingleton<IProductImportService,
ProductImportService>();
```

Если использовать `AddScoped`, то при переходе с формы импорта на список товаров данные могут потеряться, потому что каждый HTTP-запрос — это новый scope.

3.3. Модель товара для варианта 1 «Электроника»

Класс `Product` должен содержать базовые поля и дополнительные поля варианта 1.

```
public class Product
{
    public string Sku { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string Currency { get; set; } = "RUB";
    public string CategoryName { get; set; } = "Без категории";
    public int StockQuantity { get; set; }
    public string ImageUrl { get; set; } = string.Empty;
    public bool IsActive { get; set; }

    // Вариант 1: Электроника
    public string Brand { get; set; } = string.Empty;
    public int WarrantyMonths { get; set; }
    public string ModelNumber { get; set; } = string.Empty;
}
```

Правила нормализации значений:

Поле	Правило
`Sku`	trim, не пустое
`Name`	trim, не пустое
`Description`	может быть пустым
`Price`	decimal, больше 0

Поле	Правило
`Currency`	если пусто — `"RUB"`; иначе trim и можно привести к верхнему регистру
`CategoryName`	если пусто — `"Без категории"`
`StockQuantity`	int, не меньше 0
`ImageUrl`	может быть пустым
`IsActive`	true/false/1/0
`Brand`	trim, не пустое
`WarrantyMonths`	int от 0 до 60 включительно
`ModelNumber`	может быть пустым

3.4. Модель результата импорта

```
public class ImportResult
{
    public int TotalRows { get; set; }
    public int ImportedCount { get; set; }
    public List<ImportError> Errors { get; set; } = new();

    public int ErrorCount => Errors.Count;
}

public class ImportError
{
    public int RowNumber { get; set; }
    public string Message { get; set; } = string.Empty;
}
```

Семантика полей:

Поле	Значение
`TotalRows`	количество непустых строк данных после заголовка, которые были обработаны
`ImportedCount`	количество товаров, успешно добавленных в репозиторий
`Errors`	список всех ошибок; одна строка может дать несколько ошибок
`ErrorCount`	количество элементов в списке `Errors`

Номер строки:

- первая физическая строка файла — 1;

- если заголовок на первой строке, то первая строка данных имеет номер 2;
- для файловых ошибок можно использовать `RowNumber = 0`.

3.5. Интерфейс репозитория в памяти

```
public interface IProductRepository
{
    void Add(Product product);
    IReadOnlyList<Product> GetAll();
    Product? GetBySku(string sku);
    int Count { get; }
    void Clear();
}
```

Метод `Clear()` не обязателен для задания, но удобен для тестов и повторного запуска сценариев.

3.6. Реализация репозитория в памяти

Класс должен хранить товары в коллекции в памяти.

Рекомендуемая реализация:

```
public class InMemoryProductRepository : IProductRepository
{
    private readonly List<Product> _products = new();
    private readonly object _lock = new();

    public int Count
    {
        get
        {
            lock (_lock)
            {
                return _products.Count;
            }
        }
    }

    public void Add(Product product)
    {
        if (product == null)
            throw new ArgumentNullException(nameof(product));

        lock (_lock)
        {
```

```
            _products.Add(product);
        }
    }

    public IReadOnlyList<Product> GetAll()
    {
        lock (_lock)
        {
            return _products.ToList();
        }
    }

    public Product? GetBySku(string sku)
    {
        if (string.IsNullOrWhiteSpace(sku))
            return null;

        lock (_lock)
        {
            return _products.FirstOrDefault(p =>
                string.Equals(p.Sku, sku.Trim(),
StringComparison.OrdinalIgnoreCase));
        }
    }

    public void Clear()
    {
        lock (_lock)
        {
            _products.Clear();
        }
    }
}
```

Требования к поведению:

1. `Add` не должен падать при повторном добавлении, если дубликат не проверяется на уровне сервиса.
2. `GetAll` должен возвращать снапшот списка, чтобы внешнее изменение коллекции не влияло на внутреннюю.
3. `GetBySku` должен искать SKU без учета регистра и пробелов.
4. Репозиторий должен быть thread-safe, потому что будет singleton в ASP.NET Core.

3.7. Интерфейс сервиса импорта

```
public interface IProductImportService
{
```

```
    ImportResult ImportFromCsv(string filePath);
    ImportResult ImportFromLines(IEnumerable<string> lines);
}
```

Метод `ImportFromLines` нужен для unit-тестов, чтобы не зависеть от файловой системы.

3.8. Правила валидации CSV для варианта 1 «Электроника»

Обязательные заголовки:

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
```

Заголовки можно искать без учета регистра и пробелов.

Таблица валидации

Поле	Правило	Сообщение об ошибке
`sku`	обязательно, не пустое после trim	`Поле sku не может быть пустым`
`name`	обязательно, не пустое после trim	`Поле name не может быть пустым`
`description`	может быть пустым	—
`price`	число, больше 0	`Поле price должно быть числом больше 0`
`currency`	если пусто — использовать `RUB`	—
`category_name`	если пусто — использовать `Без категории`	—
`stock`	целое число, не меньше 0	`Остаток на складе должен быть целым числом не меньше 0`
`image_url`	может быть пустым	—
`is_active`	`true`, `false`, `1`, `0` без учета регистра	`Поле is_active должно иметь значение true, false, 1 или 0`
`brand`	обязательно, не пустое после trim	`Поле brand обязательно для варианта «Электроника»`
`warranty_months`	целое число от 0 до	`Гарантия должна быть целым числом от 0 до`

Поле	Правило	Сообщение об ошибке
	60 включительно	`60 месяцев`
`model_number`	может быть пустым	—

Дополнительные сообщения:

Ситуация	Сообщение
Файл не найден	`Файл не найден`
Пустой файл или нет строк данных	`Данные для импорта отсутствуют`
Неверный заголовок CSV	`Неверные заголовки CSV`
Неверное количество полей в строке	`Неверное количество полей в строке`
Дубликат SKU	`Артикул товара уже существует`

3.9. Поведение сервиса импорта

Класс:

```
public class ProductImportService : IProductImportService
{
    private readonly IProductRepository _repository;

    public ProductImportService(IProductRepository repository)
    {
        _repository = repository ?? throw new
ArgumentNullException(nameof(repository));
    }

    public ImportResult ImportFromCsv(string filePath)
    {
        // 1. Проверить существование файла.
        // 2. Прочитать строки.
        // 3. Вызвать ImportFromLines(lines).
    }

    public ImportResult ImportFromLines(IEnumerable<string> lines)
    {
        // Основная логика импорта.
    }
}
```

Алгоритм `ImportFromCsv`

1. Если `filePath` пустой:

- вернуть результат с ошибкой:

  `Файл не найден`

2. Проверить существование файла:

   ```
   File.Exists(filePath)
   ```

3. Если файл не существует:
   - `TotalRows = 0;`
   - `ImportedCount = 0;`
   - добавить ошибку с `RowNumber = 0`:

     `Файл не найден`

4. Прочитать строки файла:

   ```
   File.ReadAllLines(filePath)
   ```

5. Обернуть чтение в `try/catch` для `IOException`, `UnauthorizedAccessException`.
6. Передать строки в метод `ImportFromLines`.

Алгоритм `ImportFromLines`

1. Создать объект результата:

```
var result = new ImportResult();
```

2. Если список строк пуст или все строки пустые:
   - добавить ошибку:

     `Данные для импорта отсутствуют`

   - вернуть результат.
3. Найти первую непустую строку — это заголовок.
4. Определить физический номер строки заголовка:
   - если первая строка файла — заголовок, то `headerRow = 1`;
   - если перед заголовком есть пустые строки, использовать их реальный номер.
5. Разобрать заголовок на поля с помощью парсера CSV.
6. Проверить наличие обязательных колонок:

```
sku,name,description,price,currency,category_name,stock,image_url,is_a
ctive,brand,warranty_months,model_number
```

7. Если хотя бы одна колонка отсутствует:
    - добавить ошибку с номером строки заголовка:

      `Неверные заголовки CSV`

    - вернуть результат.

8. Создать словарь соответствия имени колонки и индекса поля:

```
var columnMap = new Dictionary<string,
int>(StringComparer.OrdinalIgnoreCase);
```

Правила:

- имена колонок trim;
- поиск без учета регистра;
- если колонка встречается несколько раз, использовать первое вхождение;
- дополнительные колонки можно игнорировать.

9. Определить ожидаемое количество полей — это количество полей в заголовке.

10. Создать множество успешно импортированных SKU для текущего файла:

```
var importedSkus = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);
```

Это нужно, чтобы корректно обрабатывать дубликаты внутри одного CSV-файла.

11. Перебрать строки после заголовка.

Для каждой строки:

11.1. Пустые строки

Если строка пустая или содержит только пробелы:

- пропустить;
- не увеличивать `TotalRows`;
- не добавлять ошибку.

11.2. Непустая строка данных

Увеличить:

```
result.TotalRows++;
```

Определить физический номер строки:

```
int rowNumber = index + 1;
```

Разобрать строку на поля с помощью парсера CSV.

Если количество полей не равно ожидаемому:

- добавить ошибку:

  `Неверное количество полей в строке`

- перейти к следующей строке.

11.3. Извлечение значений по колонкам

Создать локальную функцию или метод:

```
string GetValue(string columnName)
{
    if (columnMap.TryGetValue(columnName, out int index) && index <
values.Count)
        return values[index];

    return string.Empty;
}
```

Значения должны быть уже trimmed парсером.

11.4. Валидация полей

Создать локальный список ошибок для текущей строки:

```
var rowErrors = new List<ImportError>();
```

Проверять поля в следующем порядке:

1. `sku`
2. `name`
3. `price`
4. `stock`
5. `is_active`
6. `brand`
7. `warranty_months`

Примеры проверок:

```
string sku = GetValue("sku").Trim();

if (string.IsNullOrWhiteSpace(sku))
{
    rowErrors.Add(new ImportError
```

```
    {
        RowNumber = rowNumber,
        Message = "Поле sku не может быть пустым"
    });
}

string name = GetValue("name").Trim();

if (string.IsNullOrWhiteSpace(name))
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Поле name не может быть пустым"
    });
}
```

Для цены:

```
decimal price;
bool priceValid = TryParseDecimal(GetValue("price"), out price) &&
price > 0;

if (!priceValid)
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Поле price должно быть числом больше 0"
    });
}
```

Для остатка:

```
int stock;
bool stockValid = TryParseInt(GetValue("stock"), out stock) && stock
>= 0;

if (!stockValid)
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Остаток на складе должен быть целым числом не
меньше 0"
    });
}
```

Для is_active:

```
bool isActive;
bool activeValid = TryParseBool(GetValue("is_active"), out isActive);

if (!activeValid)
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Поле is_active должно иметь значение true, false, 1
или 0"
    });
}
```

Для бренда:

```
string brand = GetValue("brand").Trim();

if (string.IsNullOrWhiteSpace(brand))
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Поле brand обязательно для варианта «Электроника»"
    });
}
```

Для гарантии:

```
int warrantyMonths;
bool warrantyValid = TryParseInt(GetValue("warranty_months"), out
warrantyMonths)
                     && warrantyMonths >= 0
                     && warrantyMonths <= 60;

if (!warrantyValid)
{
    rowErrors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Гарантия должна быть целым числом от 0 до 60
месяцев"
    });
}
```

11.5. Если есть ошибки валидации

Если `rowErrors.Count > 0`:

- добавить все ошибки в `result.Errors`;
- не добавлять товар в репозиторий;
- перейти к следующей строке.

Пример:

```
if (rowErrors.Count > 0)
{
    result.Errors.AddRange(rowErrors);
    continue;
}
```

11.6. Проверка дубликата SKU

Проверять дубликат только если все остальные поля корректны.

Условие дубликата:

```
bool isDuplicate = _repository.GetBySku(sku) != null ||
importedSkus.Contains(sku);
```

Если дубликат найден:

- добавить ошибку:

  ```
  Артикул товара уже существует
  ```

- не добавлять товар;
- перейти к следующей строке.

Пример:

```
if (isDuplicate)
{
    result.Errors.Add(new ImportError
    {
        RowNumber = rowNumber,
        Message = "Артикул товара уже существует"
    });

    continue;
}
```

11.7. Создание объекта `Product`

Если ошибок нет и SKU не дубликат:

```
var product = new Product
{
    Sku = sku,
    Name = name,
    Description = GetValue("description").Trim(),
    Price = price,
    Currency = NormalizeCurrency(GetValue("currency")),
    CategoryName = NormalizeCategory(GetValue("category_name")),
    StockQuantity = stock,
```

```
    ImageUrl = GetValue("image_url").Trim(),
    IsActive = isActive,
    Brand = brand,
    WarrantyMonths = warrantyMonths,
    ModelNumber = GetValue("model_number").Trim()
};
```

Методы нормализации:

```
private static string NormalizeCurrency(string value)
{
    if (string.IsNullOrWhiteSpace(value))
        return "RUB";

    return value.Trim().ToUpperInvariant();
}

private static string NormalizeCategory(string value)
{
    if (string.IsNullOrWhiteSpace(value))
        return "Без категории";

    return value.Trim();
}
```

11.8. Добавление товара в репозиторий

```
_repository.Add(product);
importedSkus.Add(sku);
result.ImportedCount++;
```

Финальная обработка результата

После обработки всех строк:

Если `result.TotalRows == 0` и ошибок еще нет:

- добавить ошибку:

  `Данные для импорта отсутствуют`

Пример:

```
if (result.TotalRows == 0 && result.Errors.Count == 0)
{
    result.Errors.Add(new ImportError
    {
        RowNumber = 0,
        Message = "Данные для импорта отсутствуют"
    });
}
```

Вернуть `result`.

3.10. Парсер CSV-строк

Реализуйте класс:

```
public static class CsvLineParser
{
    public static IReadOnlyList<string> Parse(string line)
    {
        // Разбор строки CSV с поддержкой кавычек.
    }
}
```

Требования:

1. Поддерживать обычные поля, разделенные запятой.
2. Поддерживать пустые поля:

   ```
   1011,Блок питания Corsair
   CV550,,4990,RUB,Компьютеры,9,,true,Corsair,60,CV550
   ```

3. Поддерживать кавычки:

   ```
   1012,"Ноутбук, игровой",Тестовая строка с запятой в
   описании,59990,RUB,Компьютеры,1,/images/notebook.jpg,true,ASUS,36
   ,VivoBook
   ```

4. Внутри кавычек запятые не должны разделять поле.
5. Двойные кавычки внутри кавычек должны экранироваться:

   ```
   "Название ""Special"""
   ```

   должно превратиться в:

   ```
   Название "Special"
   ```

6. После разбора каждое поле нужно trim’нуть.

Если не хотите реализовывать полный RFC 4180, можно использовать упрощенный парсер, но тогда тесты с кавычками и запятыми внутри полей будут необязательными. Для учебного задания лучше реализовать базовую поддержку кавычек.

3.11. Контроллер `ProductsController`

Контроллер должен быть тонким: бизнес-логика импорта находится в сервисе.

```
public class ProductsController : Controller
{
    private readonly IProductRepository _repository;
    private readonly IProductImportService _importService;
    private readonly IWebHostEnvironment _environment;

    public ProductsController(
        IProductRepository repository,
        IProductImportService importService,
        IWebHostEnvironment environment)
    {
        _repository = repository ?? throw new
ArgumentNullException(nameof(repository));
        _importService = importService ?? throw new
ArgumentNullException(nameof(importService));
        _environment = environment ?? throw new
ArgumentNullException(nameof(environment));
    }

    [HttpGet]
    public ActionResult Index()
    {
        var viewModel = new ProductsIndexViewModel
        {
            Products = _repository.GetAll()
        };

        return View(viewModel);
    }

    [HttpGet]
    public ActionResult Import()
    {
        return View(new ImportViewModel());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Import(string filePath)
    {
        var resolvedPath = ResolveFilePath(filePath);

        if (string.IsNullOrWhiteSpace(resolvedPath))
        {
            return View("Import", new ImportViewModel
```

```
            {
                FilePath = filePath ?? string.Empty,
                Result = new ImportResult
                {
                    Errors =
                    {
                        new ImportError
                        {
                            RowNumber = 0,
                            Message = "Укажите путь к CSV-файлу"
                        }
                    }
                }
            });
        }

        var result = _importService.ImportFromCsv(resolvedPath);

        return View("Import", new ImportViewModel
        {
            FilePath = filePath ?? string.Empty,
            Result = result
        });
    }

    private string? ResolveFilePath(string input)
    {
        if (string.IsNullOrWhiteSpace(input))
            return null;

        var trimmed = input.Trim();

        if (Path.IsPathRooted(trimmed))
            return trimmed;

        if (trimmed.StartsWith("/"))
            trimmed = trimmed.TrimStart('/');

        return Path.Combine(_environment.WebRootPath, trimmed);
    }
}
```

Это позволяет в форме импорта указывать:

`/csv/sample_products.csv`

или полный путь:

`C:\data\products.csv`

3.12. View-модели

`ImportViewModel`

```
public class ImportViewModel
{
    public string FilePath { get; set; } = string.Empty;
    public ImportResult? Result { get; set; }
}
```

`ProductsIndexViewModel`

```
public class ProductsIndexViewModel
{
    public IReadOnlyList<Product> Products { get; set; } = new
List<Product>();

    public int TotalCount => Products.Count;

    public decimal TotalValue => Products.Sum(p => p.Price);
}
```

3.13. Представление `Views/Products/Import.cshtml`

Минимальное содержимое:

```
@model ImportViewModel
@{
    ViewData["Title"] = "Импорт товаров";
}

<h1>Импорт товаров из CSV</h1>

<form method="post" asp-controller="Products" asp-action="Import">
    @Html.AntiForgeryToken()

    <div class="form-group">
        <label for="filePath">Путь к CSV-файлу</label>
        <input type="text"
               id="filePath"
               name="filePath"
               value="@Model.FilePath"
               placeholder="/csv/sample_products.csv" />
    </div>

    <button type="submit" class="btn btn-primary">Импортировать
товары</button>
</form>
```

```
@if (Model.Result != null)
{
    <h2>Результат импорта</h2>

    <table class="table table-bordered">
        <thead>
            <tr>
                <th>Показатель</th>
                <th>Значение</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Всего строк</td>
                <td>@Model.Result.TotalRows</td>
            </tr>
            <tr>
                <td>Успешно импортировано</td>
                <td>@Model.Result.ImportedCount</td>
            </tr>
            <tr>
                <td>Ошибки</td>
                <td>@Model.Result.ErrorCount</td>
            </tr>
        </tbody>
    </table>

    @if (Model.Result.Errors.Count > 0)
    {
        <h3>Список ошибок</h3>

        <table class="table table-bordered">
            <thead>
                <tr>
                    <th>Строка</th>
                    <th>Описание ошибки</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var error in Model.Result.Errors)
                {
                    <tr>
                        <td>@error.RowNumber</td>
                        <td>@error.Message</td>
                    </tr>
                }
            </tbody>
        </table>
    }
```

```
    <a asp-action="Index" class="btn btn-success">Перейти к списку
товаров</a>
}
```

3.14. Представление `Views/Products/Index.cshtml`

Минимальное содержимое:

```
@model ProductsIndexViewModel
@{
    ViewData["Title"] = "Список товаров";
}

<h1>Товары</h1>

<p>Всего товаров: @Model.TotalCount</p>
<p>Общая стоимость: @Model.TotalValue.ToString("N0") RUB</p>

@if (Model.Products.Count == 0)
{
    <div class="alert alert-info">
        Товары еще не импортированы.
    </div>
}
else
{
    <table class="table table-bordered">
        <thead>
            <tr>
                <th>Артикул</th>
                <th>Название</th>
                <th>Описание</th>
                <th>Цена</th>
                <th>Валюта</th>
                <th>Категория</th>
                <th>Остаток</th>
                <th>Активен</th>
                <th>Бренд</th>
                <th>Гарантия, мес.</th>
                <th>Модель</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var product in Model.Products)
            {
                <tr>
                    <td>@product.Sku</td>
```

```
                    <td>@product.Name</td>
                    <td>@product.Description</td>
                    <td>@product.Price.ToString("N0")</td>
                    <td>@product.Currency</td>
                    <td>@product.CategoryName</td>
                    <td>@product.StockQuantity</td>
                    <td>@(product.IsActive ? "Да" : "Нет")</td>
                    <td>@product.Brand</td>
                    <td>@product.WarrantyMonths</td>
                    <td>@product.ModelNumber</td>
                </tr>
            }
        </tbody>
    </table>
}

<a asp-action="Import" class="btn btn-primary">Импортировать
товары</a>
```

---

4. План тестирования

Тесты нужно писать для:

1. `InMemoryProductRepository`
2. `CsvLineParser`
3. `ProductImportService`
4. Опционально — `ProductsController`

Ниже приведены обязательные сценарии.

4.1. Тесты репозитория

Тест 1: Добавление товара

Дано:

- новый `InMemoryProductRepository`;
- объект `Product`.

Когда:

- вызван `Add(product)`.

Тогда:

- `GetAll()` содержит этот товар;

- `Count == 1`.

Пример ожидаемого кода теста:

```
[Fact]
public void Add_AddsProduct_GetAllContainsIt()
{
    var repository = new InMemoryProductRepository();

    var product = new Product
    {
        Sku = "1001",
        Name = "Ноутбук Lenovo IdeaPad 3",
        Price = 45990m,
        Brand = "Lenovo"
    };

    repository.Add(product);

    var all = repository.GetAll();

    Assert.Single(all);
    Assert.Equal("1001", all[0].Sku);
}
```

Тест 2: Поиск по SKU

Дано:

- репозиторий с товаром `SKU = "1001"`.

Когда:

- вызван `GetBySku("1001")`.

Тогда:

- возвращается этот товар.

Тест 3: Поиск по SKU без учета регистра и пробелов

Дано:

- репозиторий с товаром `SKU = "1001"`.

Когда:

- вызван `GetBySku(" 1001 ")` или `GetBySku("1001")`.

Тогда:

- товар находится.

Если вы решите использовать точное сравнение, этот тест можно убрать. Но для учебного задания лучше использовать `StringComparison.OrdinalIgnoreCase`.

Тест 4: Пустой репозиторий

Дано:

- новый репозиторий.

Тогда:

- `Count == 0`;
- `GetAll()` пуст;
- `GetBySku("1001")` возвращает `null`.

4.2. Тесты парсера CSV

Тест 1: Простая строка без кавычек

Вход:

```
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad3-15
```

Ожидаемое количество полей: `12`.

Первое поле:

```
1001
```

Тест 2: Пустые поля

Вход:

```
1011,Блок питания Corsair CV550,,4990,RUB,Компьютеры,9,,true,Corsair,60,CV550
```

Ожидаемое количество полей: `12`.

Третье поле должно быть пустым.

Восьмое поле должно быть пустым.

Тест 3: Поле с запятой внутри кавычек

Вход:

```
1012,"Ноутбук, игровой",Тестовая
строка,59990,RUB,Компьютеры,1,/images/notebook.jpg,true,ASUS,36,VivoBo
ok
```

Ожидаемое количество полей: 12.

Второе поле должно быть:

```
Ноутбук, игровой
```

Тест 4: Экранированные кавычки

Вход:

```
1013,"Название
""Special""",Описание,100,RUB,Категория,5,/img.jpg,true,Brand,12,Model
```

Второе поле должно быть:

```
Название "Special"
```

4.3. Тесты сервиса импорта

Для большинства тестов используйте метод:

```
ImportResult ImportFromLines(IEnumerable<string> lines);
```

Это позволит не создавать файлы на диске.

Общий заголовок для тестов:

```
private const string Header =
"sku,name,description,price,currency,category_name,stock,image_url,is_
active,brand,warranty_months,model_number";
```

Тест 1: Успешный импорт одной корректной строки

Входные данные:

```
var lines = new[]
{
    Header,
```

```
    "1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad
3-15"
};
```

Дано:

- новый `InMemoryProductRepository`;
- новый `ProductImportService(repository)`.

Когда:

- вызван `service.ImportFromLines(lines)`.

Тогда:

```
Assert.Equal(1, result.TotalRows);
Assert.Equal(1, result.ImportedCount);
Assert.Empty(result.Errors);
```

И в репозитории должен быть товар:

```
var product = repository.GetBySku("1001");

Assert.NotNull(product);
Assert.Equal("Ноутбук Lenovo IdeaPad 3", product.Name);
Assert.Equal(45990m, product.Price);
Assert.Equal("Lenovo", product.Brand);
Assert.Equal(24, product.WarrantyMonths);
Assert.Equal("IdeaPad3-15", product.ModelNumber);
```

Тест 2: Успешный импорт нескольких строк

Входные данные:

- заголовок;
- две корректные строки.

Ожидаемый результат:

```
Assert.Equal(2, result.TotalRows);
Assert.Equal(2, result.ImportedCount);
Assert.Empty(result.Errors);
Assert.Equal(2, repository.Count);
```

Тест 3: Файл не найден

Дано:

- несуществующий путь:

  ```
  var path = Path.Combine(Path.GetTempPath(),
  Guid.NewGuid().ToString() + ".csv");
  ```

Когда:

- вызван `service.ImportFromCsv(path)`.

Тогда:

```
Assert.Equal(0, result.TotalRows);
Assert.Equal(0, result.ImportedCount);
Assert.Single(result.Errors);
Assert.Contains("Файл не найден", result.Errors[0].Message);
```

Тест 4: Пустой файл

Создайте пустой временный файл:

```
var path = Path.Combine(Path.GetTempPath(), Guid.NewGuid().ToString()
+ ".csv");
File.WriteAllText(path, string.Empty);
```

Когда:

- вызван `service.ImportFromCsv(path)`.

Тогда:

```
Assert.Equal(0, result.TotalRows);
Assert.Equal(0, result.ImportedCount);
Assert.Single(result.Errors);
Assert.Contains("Данные для импорта отсутствуют",
result.Errors[0].Message);
```

Тест 5: Только заголовок без данных

Входные данные:

```
var lines = new[] { Header };
```

Ожидаемый результат:

```
Assert.Equal(0, result.TotalRows);
Assert.Equal(0, result.ImportedCount);
Assert.Single(result.Errors);
Assert.Contains("Данные для импорта отсутствуют",
result.Errors[0].Message);
```

Тест 6: Неверный заголовок CSV

Входные данные:

```
var lines = new[]
{

"sku,name,description,price,currency,category_name,stock,image_url,is_
active"
};
```

Ожидаемый результат:

- `TotalRows == 0;`
- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Неверные заголовки CSV`

Тест 7: Неверное количество полей в строке данных

Входные данные:

```
var lines = new[]
{
    Header,
    "1001,Ноутбук Lenovo IdeaPad 3"
};
```

Ожидаемый результат:

- `TotalRows == 1;`
- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Неверное количество полей в строке`

Тест 8: Пустой SKU

Входные данные:

```
var lines = new[]
{
    Header,
    ",Ноутбук Lenovo IdeaPad 3,Бюджетный
ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaP
ad3-15"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле sku не может быть пустым`

Тест 9: Пустое название товара

Входные данные:

```
var lines = new[]
{
    Header,
    "1001,,Бюджетный
ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaP
ad3-15"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле name не может быть пустым`

Тест 10: Неверная цена — отрицательное значение

Входные данные:

```
var lines = new[]
{
    Header,
    "1003,Клавиатура A4Tech Bloody V3M,Механическая игровая
клавиатура,-500,RUB,Периферия,7,/images/keyboard.jpg,true,A4Tech,24,V3
```

```
M"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле price должно быть числом больше 0`

Тест 11: Неверная цена — нечисловое значение

Входные данные:

```
var lines = new[]
{
    Header,
    "1003,Клавиатура A4Tech Bloody V3M,Механическая игровая
клавиатура,abc,RUB,Периферия,7,/images/keyboard.jpg,true,A4Tech,24,V3M
"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле price должно быть числом больше 0`

Тест 12: Отрицательный остаток

Входные данные:

```
var lines = new[]
{
    Header,
    "1004,Монитор Dell S2722QC,Игровой монитор 27
дюймов,29990,RUB,Компьютеры,-5,/images/monitor.jpg,true,Dell,24,S2722Q
C"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Остаток на складе должен быть целым числом не меньше 0`

Тест 13: Неверное значение `is_active`

Входные данные:

```
var lines = new[]
{
    Header,
    "1007,Webcam Logitech C920,Веб-камера Full
HD,4990,RUB,Периферия,6,/images/webcam.jpg,yes,Logitech,24,C920"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле is_active должно иметь значение true, false, 1 или 0`

Тест 14: Отсутствует бренд

Входные данные:

```
var lines = new[]
{
    Header,
    "1005,Видеокарта NVIDIA GeForce RTX
3060,,24990,RUB,Компьютеры,4,/images/gpu.jpg,true,,36,RTX3060"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Поле brand обязательно для варианта «Электроника»`

Тест 15: Гарантия меньше нуля

Входные данные:

```
var lines = new[]
{
    Header,
    "1006,Накопитель SSD Samsung 870 EVO 1TB,Внутренний твердотельный
накопитель,5990,RUB,Компьютеры,12,/images/ssd.jpg,true,Samsung,-
1,870EVO-1T"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Гарантия должна быть целым числом от 0 до 60 месяцев`

Тест 16: Гарантия больше 60

Входные данные:

```
var lines = new[]
{
    Header,
    "1006,Накопитель SSD Samsung 870 EVO 1TB,Внутренний твердотельный
накопитель,5990,RUB,Компьютеры,12,/images/ssd.jpg,true,Samsung,61,870E
VO-1T"
};
```

Ожидаемый результат:

- `ImportedCount == 0;`
- есть ошибка;
- сообщение содержит:

  `Гарантия должна быть целым числом от 0 до 60 месяцев`

Тест 17: Дубликат SKU внутри одного файла

Входные данные:

```
var lines = new[]
{
    Header,
    "1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad
3-15",
    "1001,Ноутбук ASUS VivoBook,Другой
ноутбук,52990,RUB,Компьютеры,7,/images/asus.jpg,true,ASUS,24,VivoBook"
};
```

Ожидаемый результат:

```
Assert.Equal(2, result.TotalRows);
Assert.Equal(1, result.ImportedCount);
Assert.Single(result.Errors);
Assert.Contains("Артикул товара уже существует",
result.Errors[0].Message);
```

В репозитории должен остаться первый товар:

```
var product = repository.GetBySku("1001");

Assert.NotNull(product);
Assert.Equal("Ноутбук Lenovo IdeaPad 3", product.Name);
```

Тест 18: Дубликат SKU с уже существующим товаром в репозитории

Дано:

- в репозиторий заранее добавлен товар с `SKU = "1001"`.

Входные данные:

```
var lines = new[]
{
    Header,
    "1001,Ноутбук ASUS VivoBook,Другой
ноутбук,52990,RUB,Компьютеры,7,/images/asus.jpg,true,ASUS,24,VivoBook"
};
```

Ожидаемый результат:

- новый товар не добавлен;
- `repository.Count == 1`;
- есть ошибка:

  `Артикул товара уже существует`

Тест 19: Несколько ошибок в одной строке

Входные данные:

```
var lines = new[]
{
    Header,
    ",,Некорректная
строка,-10,RUB,Компьютеры,-5,/images/x.jpg,yes,,61"
};
```

Ожидаемый результат:

- `ImportedCount == 0`;
- количество ошибок должно быть не меньше 4;
- в репозитории нет товаров.

Возможные ошибки:

```
Поле sku не может быть пустым
Поле name не может быть пустым
Поле price должно быть числом больше 0
Остаток на складе должен быть целым числом не меньше 0
Поле is_active должно иметь значение true, false, 1 или 0
Поле brand обязательно для варианта «Электроника»
Гарантия должна быть целым числом от 0 до 60 месяцев
```

Тест 20: Пустые необязательные поля разрешены

Входные данные:

```
var lines = new[]
{
    Header,
    "1011,Блок питания Corsair
CV550,,4990,RUB,Компьютеры,9,,true,Corsair,60,CV550"
};
```

Ожидаемый результат:

- товар импортирован;
- `Description` пуст;
- `ImageUrl` пуст;
- ошибок нет.

Тест 21: Пустая валюта заменяется на RUB

Входные данные:

```
var lines = new[]
{
    Header,
    "1011,Блок питания Corsair
CV550,,4990,,Компьютеры,9,,true,Corsair,60,CV550"
};
```

Ожидаемый результат:

- товар импортирован;
- `product.Currency == "RUB"`.

Тест 22: Пустая категория заменяется на «Без категории»

Входные данные:

```
var lines = new[]
{
    Header,
    "1011,Блок питания Corsair
CV550,,4990,RUB,,9,,true,Corsair,60,CV550"
};
```

Ожидаемый результат:

- товар импортирован;
- `product.CategoryName == "Без категории"`.

Тест 23: Кириллица сохраняется корректно

Входные данные:

```
var lines = new[]
{
    Header,
    "1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и
учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad
3-15"
};
```

Ожидаемый результат:

```
Assert.Equal("Ноутбук Lenovo IdeaPad 3", product.Name);
Assert.Equal("Бюджетный ноутбук для работы и учебы",
```

```
product.Description);
Assert.Equal("Компьютеры", product.CategoryName);
```

4.4. Ручные тестовые сценарии для UI

После запуска приложения выполните следующие шаги:

Сценарий 1: Успешный импорт

1. Откройте приложение в браузере.
2. Перейдите на страницу импорта товаров.
3. В поле пути укажите:

   `/csv/sample_products.csv`
4. Нажмите кнопку «Импортировать товары».
5. Убедитесь, что результат содержит:
   - всего строк — 11;
   - успешно импортировано — 11;
   - ошибок — 0.
6. Перейдите на страницу списка товаров.
7. Убедитесь, что все 11 товаров отображаются в таблице.

Сценарий 2: Неверная цена

1. Создайте файл `invalid_price.csv` из раздела выше.
2. Импортируйте его через UI или укажите полный путь к файлу.
3. Убедитесь, что:
   - товар не добавлен;
   - есть ошибка по строке данных;
   - сообщение содержит:

     `Поле price должно быть числом больше 0`

Сценарий 3: Дубликат SKU

1. Импортируйте `valid_electronics.csv`.
2. Повторно импортируйте тот же файл.

3. Убедитесь, что:
    - приложение не падает;
    - товары с дублирующимися SKU пропускаются;
    - выводятся ошибки о существующем артикуле.

Сценарий 4: Пустой файл

1. Создайте пустой CSV-файл.
2. Импортируйте его.
3. Убедитесь, что:
    - приложение не падает;
    - выводится сообщение:

      `Данные для импорта отсутствуют`

Сценарий 5: Неверное значение `is_active`

1. Импортируйте файл `bad_active.csv`.
2. Убедитесь, что:
    - товар не добавлен;
    - выводится ошибка про допустимые значения `true`, `false`, `1`, `0`.

---

5. Пошаговый план разработки проекта

Ниже — практический чек-лист, по которому можно реализовать проект от начала до конца.

Шаг 1. Создать решение и проекты

Создайте:

1. ASP.NET Core MVC-проект `ECommerceCsvImport`.
2. xUnit-тестовый проект `ECommerceCsvImport.Tests`.
3. Добавьте тестовый проект в решение.
4. Добавьте ссылку из тестового проекта на основной проект.

Пример команд:

```
dotnet new sln -n ECommerceCsvImport
dotnet new mvc -n ECommerceCsvImport -o src/ECommerceCsvImport
```

```
dotnet new xunit -n ECommerceCsvImport.Tests -o
tests/ECommerceCsvImport.Tests
dotnet sln add src/ECommerceCsvImport tests/ECommerceCsvImport.Tests
cd tests/ECommerceCsvImport.Tests
dotnet add reference
../../src/ECommerceCsvImport/ECommerceCsvImport.csproj
```

Если структура без `src` и `tests`, адаптируйте пути.

Шаг 2. Добавить CSV-файлы

Создайте папки:

```
wwwroot/csv/
tests/ECommerceCsvImport.Tests/testdata/
```

Добавьте файлы:

```
wwwroot/csv/sample_products.csv
tests/ECommerceCsvImport.Tests/testdata/valid_electronics.csv
tests/ECommerceCsvImport.Tests/testdata/invalid_price.csv
tests/ECommerceCsvImport.Tests/testdata/duplicate_sku.csv
tests/ECommerceCsvImport.Tests/testdata/missing_brand.csv
tests/ECommerceCsvImport.Tests/testdata/warranty_out_of_range.csv
tests/ECommerceCsvImport.Tests/testdata/bad_active.csv
tests/ECommerceCsvImport.Tests/testdata/empty_file.csv
```

Убедитесь, что кодировка файлов — UTF-8.

Шаг 3. Реализовать модели данных

Создайте файлы:

```
Models/Product.cs
Models/ImportResult.cs
Models/ImportError.cs
Models/ImportViewModel.cs
Models/ProductsIndexViewModel.cs
```

Проверьте, что проект собирается:

```
dotnet build
```

Шаг 4. Реализовать репозиторий в памяти

Создайте файлы:

```
Services/IProductRepository.cs
Services/InMemoryProductRepository.cs
```

Реализуйте методы:

- `Add`
- `GetAll`
- `GetBySku`
- `Count`
- `Clear`

Добавьте блокировку для thread-safety.

Напишите тесты:

```
ECommerceCsvImport.Tests/RepositoryTests.cs
```

Запустите тесты:

```
dotnet test
```

Шаг 5. Реализовать парсер CSV-строк

Создайте файл:

```
Services/CsvLineParser.cs
```

Реализуйте метод:

```
public static IReadOnlyList<string> Parse(string line)
```

Требования:

- разделение по запятым;
- поддержка пустых полей;
- поддержка кавычек;
- trim значений.

Напишите тесты:

```
ECommerceCsvImport.Tests/CsvLineParserTests.cs
```

Шаг 6. Реализовать сервис импорта

Создайте файлы:

```
Services/IProductImportService.cs
Services/ProductImportService.cs
```

Реализуйте методы:

```
ImportResult ImportFromCsv(string filePath);
ImportResult ImportFromLines(IEnumerable<string> lines);
```

Внутри сервиса реализуйте:

1. проверку файла;
2. чтение строк;
3. поиск заголовка;
4. маппинг колонок;
5. разбор строк данных;
6. валидацию полей;
7. обработку дубликатов SKU;
8. создание объекта `Product`;
9. добавление товара в репозиторий;
10. формирование `ImportResult`.

Напишите тесты:

`ECommerceCsvImport.Tests/ProductImportServiceTests.cs`

Шаг 7. Зарегистрировать зависимости

В `Program.cs` добавьте:

```
builder.Services.AddSingleton<IProductRepository,
InMemoryProductRepository>();
builder.Services.AddSingleton<IProductImportService,
ProductImportService>();
```

Это критически важно для сохранения данных в памяти между запросами.

Шаг 8. Реализовать контроллер

Создайте или обновите:

`Controllers/ProductsController.cs`

Добавьте действия:

- `Index()` — список товаров;
- `Import()` GET — форма импорта;
- `Import(string filePath)` POST — запуск импорта.

Реализуйте метод `ResolveFilePath`, чтобы поддерживались пути вида:

`/csv/sample_products.csv`

и полные пути:

`C:\data\products.csv`

Шаг 9. Реализовать представления

Создайте или обновите:

```
Views/Products/Index.cshtml
Views/Products/Import.cshtml
```

На странице импорта должны быть:

- поле ввода пути к CSV;
- кнопка запуска импорта;
- таблица результата;
- список ошибок.

На странице списка товаров должна быть таблица со всеми полями товара, включая:

- бренд;
- гарантию;
- модель.

Шаг 10. Провести ручное тестирование

Выполните сценарии из раздела «Ручные тестовые сценарии для UI».

Проверьте:

1. успешный импорт `sample_products.csv`;
2. отображение товаров на странице списка;
3. обработку неверной цены;
4. обработку дубликатов SKU;
5. обработку пустого файла;
6. обработку неверного значения `is_active`.

Шаг 11. Выполнить самопроверку качества кода

Проверьте:

Критерий	Требование
Имена классов	Понятные, например `ProductImportService`
Имена методов	Отражают действие, например `ImportFromCsv`, `GetBySku`
Обработка ошибок	Приложение не падает из-за неверного CSV
Разделение ответственности	Controller не содержит бизнес-логику импорта
Читаемость	Код отформатирован, без мусора
Комментарии	Ключевые методы имеют краткие комментарии
Исключения	Перехватываются там, где это уместно
Магические числа	Константы для `RUB`, `Без категории`, границ гарантии
Git-коммиты	Сообщения осмысленные

Шаг 12. Оформить отчет по практике

Отчет должен содержать:

1. Титульный лист.
2. Цель и задачи практики.
3. Описание задания.
4. Архитектуру решения.
5. Формат CSV-файла.
6. Реализацию функциональности.
7. Результаты тестирования.
8. Выводы.
9. Приложения:
    - пример CSV;
    - скриншоты страниц;
    - список ошибок импорта;
    - фрагменты кода;
    - историю Git-коммитов.

---

6. Рекомендуемые Git-коммиты

Используйте осмысленные сообщения коммитов:

```
Add project scaffold and CSV fixtures
Implement product model and in-memory repository
Implement CSV line parser with quote support
Implement product import service validation for electronics variant
Add MVC controller and views for product import
Add unit tests for repository, parser and import service
Fix duplicate SKU handling and final code review
```

---

7. Финальный чек-лист готовности задания к защите

Перед защитой убедитесь, что выполнены следующие пункты:

- ☐ Создан ASP.NET MVC-проект `ECommerceCsvImport`.
- ☐ Реализован класс `Product` с полями варианта 1 «Электроника».
- ☐ Реализовано хранение товаров в памяти.
- ☐ Репозиторий зарегистрирован как singleton.
- ☐ Реализовано чтение CSV-файла.
- ☐ Реализован парсер CSV с поддержкой пустых полей и кавычек.
- ☐ Реализована валидация базовых полей.
- ☐ Реализована валидация `brand`, `warranty_months`, `model_number`.
- ☐ Реализована обработка дубликатов SKU.
- ☐ Создана страница импорта товаров.
- ☐ Создана страница списка товаров.
- ☐ Модуль успешно импортирует корректный CSV-файл.
- ☐ Модуль корректно обрабатывает некорректный CSV-файл.
- ☐ Приложение не падает при пустом файле, неверной цене, дубликате SKU и неверном `is_active`.
- ☐ Код размещен в Git.
- ☐ Написаны unit-тесты для репозитория, парсера и сервиса импорта.
- ☐ Все тесты проходят успешно.
- ☐ Выполнена самопроверка качества кода.
- ☐ Подготовлены тестовые сценарии.
- ☐ Оформлен отчет по практике.
- ☐ Студент может объяснить архитектуру, валидацию и ключевые фрагменты кода.

---

8. Краткое резюме для реализации

Для самостоятельной разработки используйте такой порядок:

1. Создайте MVC-проект и тестовый проект.
2. Добавьте CSV-файл `sample_products.csv` из раздела выше.
3. Реализуйте модель `Product` с полями варианта 1 «Электроника».
4. Реализуйте `InMemoryProductRepository` как singleton.
5. Реализуйте `CsvLineParser`.
6. Реализуйте `ProductImportService` с методами:
    - `ImportFromCsv`;
    - `ImportFromLines`.
7. Добавьте валидацию:
    - обязательные поля;
    - числовые значения;
    - допустимые значения `is_active`;
    - гарантию от 0 до 60 месяцев;
    - дубликаты SKU.
8. Создайте страницы:
    - импорт товаров;
    - список товаров.
9. Напишите тесты для всех сценариев из раздела «План тестирования».
10. Проведите ручное тестирование через браузер.
11. Оформите отчет и подготовьтесь к защите.